CS233 Food Delivery System

Revised Design and Implementation

Joshua Kelly, Lincoln Wornardt, Niko Cimbur, and William Ho

Cuesta College

CS 233 (75701)

Prof. Dube

November 30, 2026

How our delivery system works

The first drawing gave us the main classes to start with. The implementation keeps that setup, but adds the file manager, menu-item objects, authentication, and the desktop screens. Both the UI and terminal version use the same `DeliverySystem`, so they follow the same rules.

Updated diagrams

- Users, food, and orders
- The app, shared system, and files
- Editable class model
- Editable app structure

The diagrams show the main fields and methods. Routine getters and the UI's small layout helpers are left out to keep them readable. `Driver` also implements `Comparable<Driver>`, and `DeliverySystem` implements `AutoCloseable`. The old drawing is kept separately so we can see what changed.

What each part does

`User` holds the account information shared by everyone. `Admin`, `Customer`, and `Driver` extend it and choose their own role and terminal options. `Driver` also keeps availability, recent ratings, and the average rating.

`Menu` looks up food by name. Each `MenuItem` has a name and a float price. `Order` keeps the customer, item prices from checkout, status, assigned driver, and the customer's rating. The customer and driver references are usernames; `DeliverySystem` checks them against the saved user objects.

`DeliverySystem` handles logins, permissions, menu changes, waiting orders, assignment, delivery updates, and ratings. Keeping those checks here means a button cannot bypass an ownership or status rule.

`NumberUtil` checks float prices and ratings and formats money to two decimal places. `FileManager` reads and writes the text files. `PasswordUtil` handles password hashing. `ActivityEntry` is one saved activity event. `Main` opens either `DeliveryApp`, which is the Swing UI, or `TerminalApp` when `--cli` is passed.

The four OOP principles

Encapsulation

The account fields, order state, and driver ratings are private. The order and rating getters return copies of their lists, so clearing a list in a screen doesn't clear the system's data. `MenuItem` and `ActivityEntry` have final fields, no setters, and final classes. This lets orders safely keep the menu items and prices they had at checkout.

An order's status is changed through methods like `startDelivery()` and `completeDelivery()`. Those methods check the previous status. `DeliverySystem` also checks the logged-in role and the assigned driver's username before it calls them. Customers can only rate or cancel their own orders.

Abstraction

`User` is abstract because a plain user account doesn't have its own dashboard. Each role has to implement `getRole()` and `viewOptions()`. The screens work through the shared delivery system instead of needing to know how order records are formatted or how the driver heap is maintained.

Inheritance

`Admin`, `Customer`, and `Driver` inherit the username, email, password check, and account methods from `User`. We don't need three copies of the login code. The driver subclass adds the information that only drivers need, like ratings and availability.

Polymorphism

The terminal keeps the logged-in account in a `User` variable and calls `user.viewOptions()`. Java runs the admin, customer, or driver implementation based on the actual object. `getRole()` works the same way. The driver pool also uses the `Comparable<Driver>` implementation when ordering drivers by average rating and then username.

Data structures and why we picked them

Table 1

Data structures used by the system

Job	Structure	Why it fits
Waiting orders	`Queue<Order>` backed by `ArrayDeque`	Adds at the back and accepts from the front, preserving FIFO order
Available drivers	`PriorityQueue<Driver>`	Keeps the highest-rated available driver at the head
Recent ratings	`ArrayDeque<Float>`	Removes the oldest rating from the front and adds the newest at the back
Menu lookup	`HashMap<String, MenuItem>`	Finds an item by its normalized name without searching the whole menu
Users and order history	`LinkedHashMap`	Looks up usernames or order numbers while keeping insertion order
Items inside an order	`ArrayList<MenuItem>`	Keeps the selected items, including repeated items for quantities
Activity page	`ArrayDeque<ActivityEntry>`	Keeps the latest 200 matching events for display; the full log stays on disk

Queue insertion and removal are amortized O(1). For `d` available drivers, checking the next driver is O(1), and adding or taking the head driver is O(log d). Updating a particular available driver's rating takes O(d) to remove that object from the heap, followed by O(log d)

to insert it again. We do that before using its changed average so the heap does not keep an outdated order. Oracle's PriorityQueue documentation explains these operation costs (Oracle, n.d.-b).

Ratings are limited to ten, so recalculating an average visits at most ten numbers. Hash-map lookups are O(1) on average. Displaying the menu sorts its items by name, which takes O(m log m) for `m` menu items. Cancelling a waiting order scans the queue, so that optional action takes O(q) for `q` waiting orders.

Order and assignment rules

1. Check that the customer is logged in and every selected item exists.
2. Create an order with a new number and the current item prices.
3. Append its placed record to `orders.txt` before adding it to the waiting queue.
4. If a driver is free, take the oldest waiting order and the highest-rated available driver. Save the accepted order and mark that driver busy.
5. Only that driver can move the order from accepted to in progress, then delivered.
6. Finishing a delivery returns the driver to the pool and tries the next waiting order.
7. The order's customer can give it one float rating from 1 to 5 after delivery, including decimals such as 4.5.
8. Keep the last ten driver ratings and update the average and assignment priority.

A new driver starts with an average of zero until they get a rating. Equal averages are resolved by username so the result is predictable. Each driver handles one active order at a time. Customers can cancel their own waiting or accepted orders until delivery starts. Cancelling an accepted order returns its driver to the available pool and assigns the oldest waiting order, if there is one. The cancelled order keeps its driver in the saved history. In-progress and delivered orders cannot be cancelled. An empty order or an order with more than 100 items is rejected.

Text files and file I/O

The working data folder has these files:

Table 2

Text file record formats

File	Record format
`Menu.txt`	`food name, price`
`customers.txt` and `admins.txt`	`username|passwordHash|email`
`drivers.txt`	`username|passwordHash|email|available|ratings`
`orders.txt`	`id|customer|status|driver|rating|driverRatings|itemCount|name|price|...`
`activity.txt`	`UTC time|role|username|event|details`

Prices, totals, ratings, and rating averages use `float`. The saved text uses decimal numbers; old whole-number ratings still load through `Float.parseFloat`. `NumberUtil` rejects non-finite numbers and prices with fractions of a cent. The cart and orders use the same total calculation, adding rounded float cents before converting back to dollars. Money is formatted with two decimal places for display and saving. Binary floats are approximate; rounding does not make them exact decimal values, especially for large amounts.

These are schemas, not sample records to paste into the files. Driver ratings are comma-separated and stored oldest first. A rating of zero on an order means it has not been rated. Each order record includes a name and price pair for every selected item.

On the first run, the program takes a lock on the folder, copies the starting menu, and creates the demo user files and empty logs. It only initializes demo data when none of the six data files exist. A partly missing data set is reported instead of being reset.

On later runs, `FileManager` reads the menu and user files as UTF-8 text. It checks record lengths, numbers, usernames, password hashes, and ratings. It then replays `orders.txt` in file order. The first record creates an order; later records replace that order's saved state after its transition is checked. Updates to existing keys do not change their insertion order. That lets the system rebuild the waiting queue in the original received order.

A rating event also stores the resulting driver-rating list. Replaying the last such event restores that list, even if the driver snapshot was not updated before the app stopped. Active orders determine which drivers are busy. New order numbers continue after the highest saved number.

Menu and user-file changes are written to a temporary file and then moved into place. An atomic move is used when the file system supports it. Orders and activity are appended instead of rewriting their history. These writes are not one multi-file database transaction: if a later write fails, the UI reports it and closes. The log replay can recover completed order appends, but unreadable or truncated records still need to be restored from a good copy.

The app validates names and usernames so separators and line breaks cannot turn one saved record into several. Two running copies cannot edit the same data folder because the file lock stays held until the app closes.

The three UIs and activity logging

The customer screen has the food menu, cart, order history, rating action, and Activity page. The driver screen has assigned deliveries, status buttons, ratings, and Activity. The admin screen has all orders, menu editing, the driver pool, account creation, and the full activity history.

The activity log records real method requests, assignment decisions, file loads, and successful saves. It shows which files changed without logging passwords, hashes, or full user-file contents. Customers and drivers can only read activity attached to their username.

Admins can read the system and all user events. Times are stored in UTC and displayed in the computer's local time.

The UI and Java code run in the same process. These are local backend calls, not HTTP requests. Swing is part of Java (Oracle, n.d.-a), which keeps the Windows and macOS versions in one language. Both the UI and terminal version use the same saved files and rules. The app works with one shared session at a time; this version is not a multi-device delivery service.

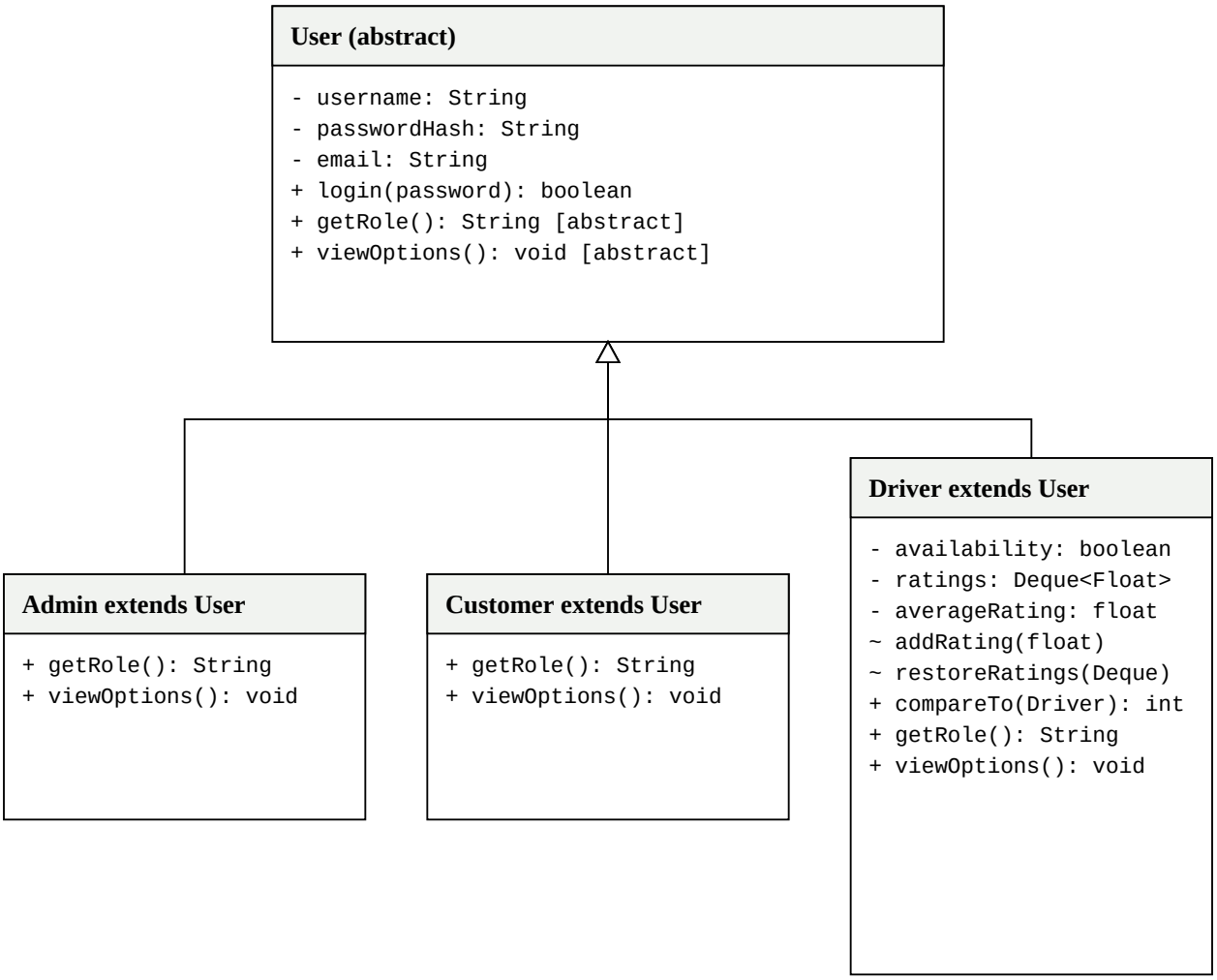
Checks so far

The automated checks cover complete orders, FIFO behavior after restart, assignment when drivers become free, rating-based reordering, the ten-rating limit, menu changes, all user roles, ownership rules, file errors, and retained activity. Separate Java processes check terminal sessions and the packaged JAR, including folder paths with spaces. Offscreen UI checks run the real button handlers and render the panels without showing a window.

The current local results are 126 backend checks, 14 terminal/JAR checks, and 32 desktop checks. A Windows/macOS/Linux Actions workflow is prepared in `.github/workflows/java.yml`. GitHub's current connection permissions do not allow that file to be uploaded, so a Windows runtime result is still pending.

Figure 1

User roles and shared account behavior

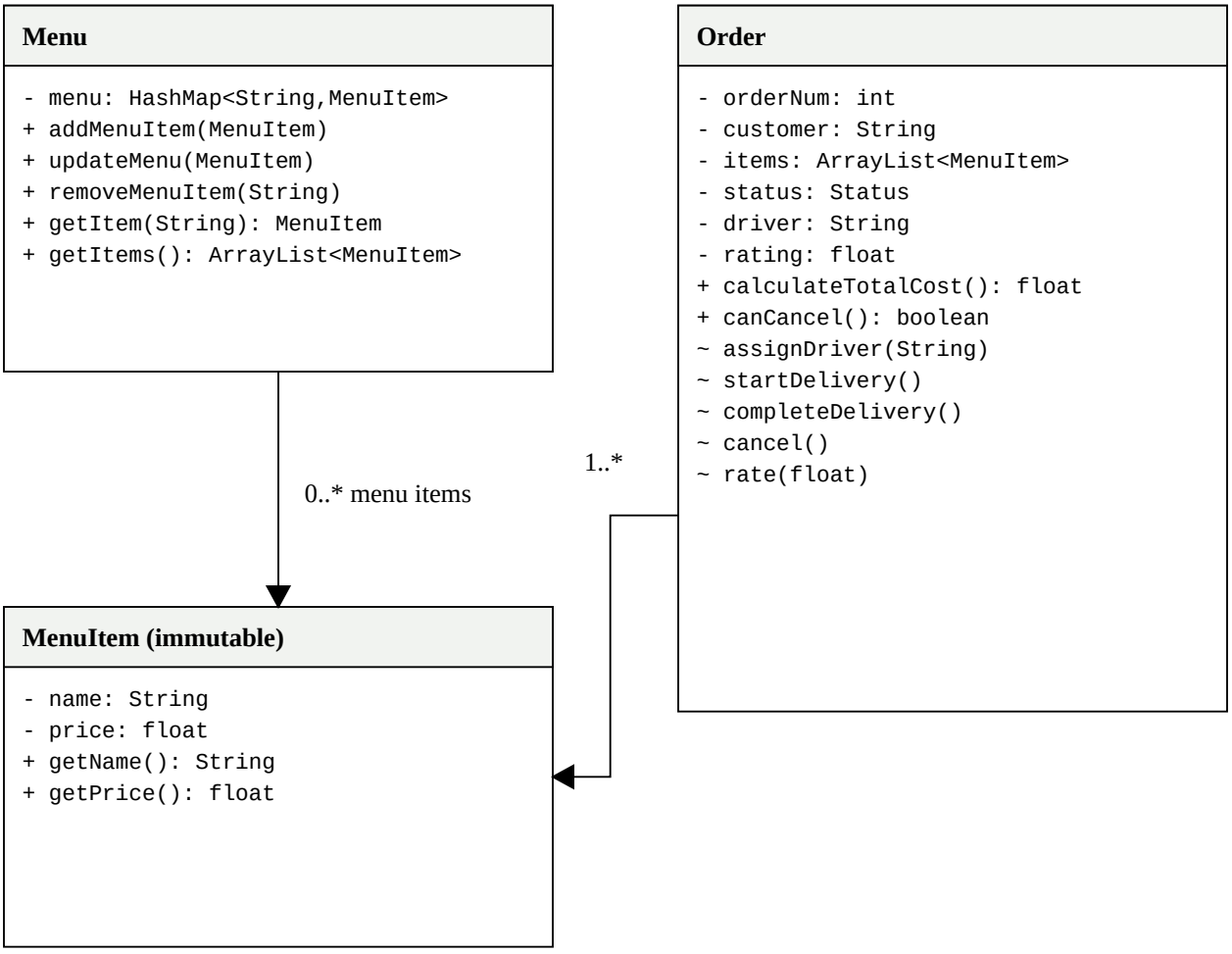


Driver also implements Comparable<Driver>.

Note. Compact class view. Full-size PNG, SVG, and editable Excalidraw diagrams are included in docs/.

Figure 2

Menu items and saved orders

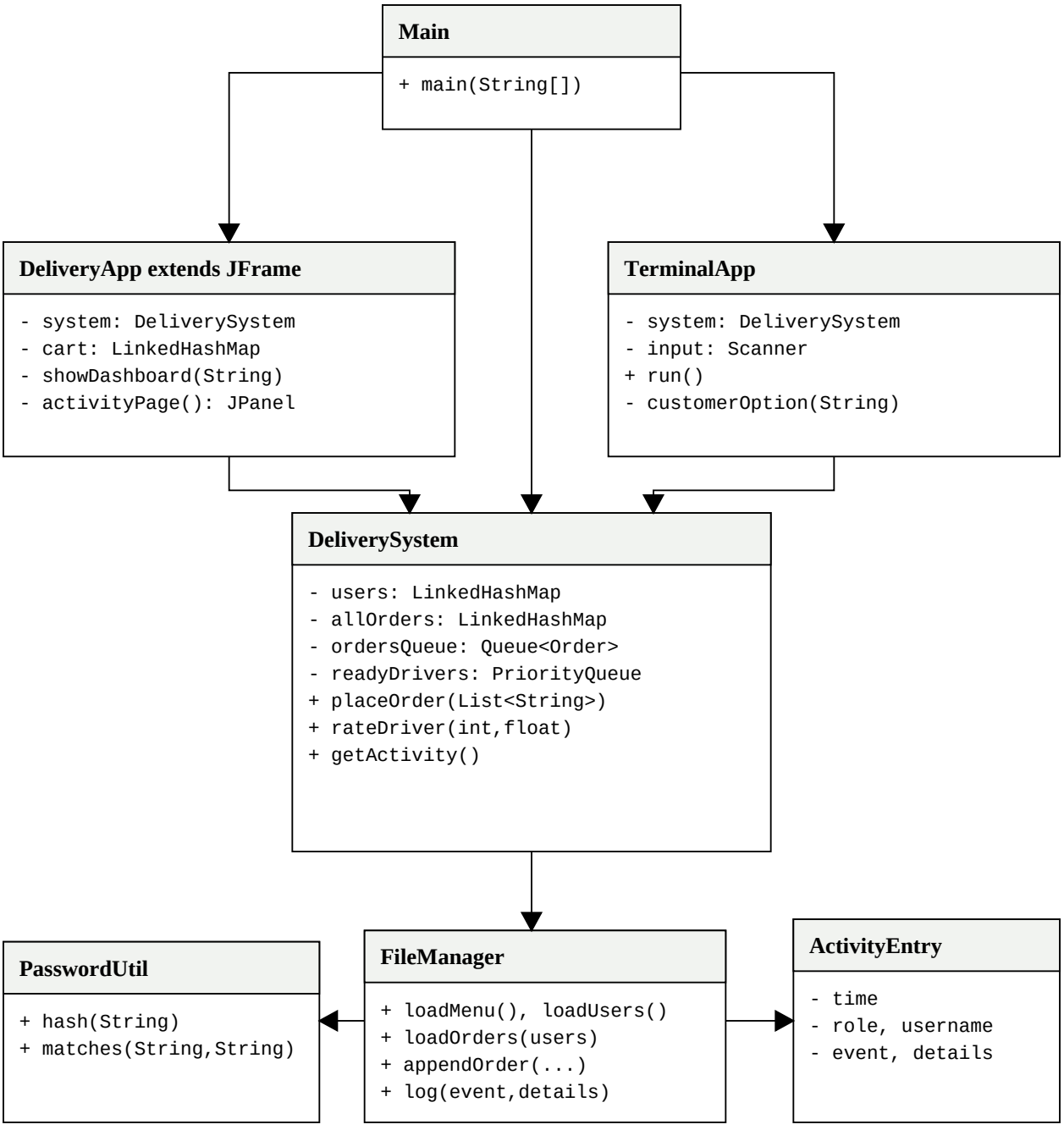


Order stores customer and driver usernames; the system validates ownership.

Note. Compact class view. Full-size PNG, SVG, and editable Excalidraw diagrams are included in docs/.

Figure 3

Interfaces, delivery rules, and persistence



Note. Compact class view. Full-size PNG, SVG, and editable Excalidraw diagrams are included in docs/.

References

Oracle. (n.d.-a). *Package javax.swing (Java SE 21 & JDK 21).* https://docs.oracle.com/en/java/javase/21/docs/api/java.desktop/javax/swing/package-summary.html

Oracle. (n.d.-b). *PriorityQueue (Java SE 17 & JDK 17).* https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/PriorityQueue.html